

DOCUMENTO DE REQUISITO

Backup e Continuidade de Operação

Projeto Integrado - PNCP Data Engine

Desenvolvido por: Lucas Ferreira, Isaac Daniel, Arthur Orange, Antônio, Guilherme, Gabriel Garcia, Priscila Maciel e João Lucas Santos Lira

GitHub: <https://github.com/Lukitas20-beep/ProjEngDados>

Campo	Descrição
Requisito avaliado	5 - Backup e continuidade de operação
Período de documentação	03-04/06
Data de avaliação da implementação	10-11/06
Sistema	Aplicação Streamlit para extração, transformação, visualização e carga de dados do PNCP
Versão do documento	1.0

1. Contexto do requisito

O PNCP Data Engine realiza extração, transformação, visualização e carga de dados obtidos a partir da API do Portal Nacional de Contratações Públicas. Nas entregas anteriores, o projeto passou a contar com autenticação de usuários, gestão segura de senhas, anonimização preventiva de informações, atendimento a requisitos de LGPD e mecanismos de disponibilidade e proteção contra ataques digitais.

A quinta entrega acrescenta controles voltados a backup e continuidade de operação. O objetivo é reduzir o risco de perda de dados locais e permitir retomada controlada em caso de falha, corrupção de banco, erro operacional ou necessidade de auditoria.

2. Objetivo do requisito

Implementar e documentar mecanismos básicos de backup, verificação de integridade, restauração controlada, retenção de cópias e diagnóstico de continuidade operacional. A entrega preserva os bancos locais usados nas entregas anteriores e inclui exportação opcional de dados do MongoDB em formato JSONL.

3. Escopo da entrega

Criação de módulo específico para backup e continuidade; backup dos bancos SQLite locais; geração de manifesto com hash SHA-256; compactação em ZIP; listagem de backups; verificação de integridade; restauração controlada com cópia anterior; exportação opcional do MongoDB; política simples de retenção; diagnóstico de continuidade; atualização do .env.example e do .gitignore.

4. Requisitos funcionais

Código	Requisito funcional	Descrição
RF01	Criar backup local	O sistema deve permitir criar backup dos bancos SQLite locais usados pela aplicação.
RF02	Gerar manifesto	Cada backup deve conter manifest.json com data, arquivos, tamanho, hash e status.
RF03	Compactar backup	O backup deve ser empacotado em arquivo ZIP.

RF04	Listar backups	A interface deve listar os backups disponíveis no diretório configurado.
RF05	Verificar integridade	O sistema deve recalcular hashes e validar o pacote gerado.
RF06	Restaurar banco local	O sistema deve restaurar bancos SQLite de forma controlada e confirmada.
RF07	Preservar cópia anterior	Antes da restauração, o banco substituído deve ser preservado.
RF08	Exportar MongoDB	O sistema deve permitir exportar uma coleção MongoDB em JSONL.
RF09	Aplicar retenção	O sistema deve remover backups antigos conforme política configurável.
RF10	Verificar continuidade	O sistema deve exibir diagnóstico de bancos locais, diretório e último backup.

5. Requisitos não funcionais

Código	Requisito não funcional	Descrição
RNF01	Confiabilidade	Backups devem conter hashes para verificação posterior.
RNF02	Recuperabilidade	A restauração deve preservar cópia anterior para reduzir risco operacional.
RNF03	Segurança operacional	Backups, bancos locais e logs não devem ser versionados no Git.
RNF04	Configurabilidade	Diretório, retenção e limites devem ser configuráveis por variáveis de ambiente.
RNF05	Manutenibilidade	A lógica de backup deve ficar concentrada em módulo próprio.
RNF06	Baixo impacto	As rotinas de backup não devem interromper o fluxo normal da aplicação.

6. Regra de negócio

Usuários autenticados podem acionar rotinas de backup e continuidade pela interface. O backup local deve incluir os bancos SQLite configurados quando eles existirem. Cada pacote deve possuir manifesto verificável. A restauração exige confirmação explícita e cria cópia de segurança do banco anterior antes de sobrescrever o arquivo.

7. Fluxo de backup e continuidade

O usuário acessa a área de Backup e continuidade; aciona a criação do backup; o sistema copia os bancos locais existentes; calcula hash SHA-256; gera manifesto; compacta os arquivos em ZIP; registra evento de segurança; lista os backups disponíveis; permite verificar integridade; e, quando confirmado, restaura bancos locais com preservação da versão anterior.

8. Controles implementados

Controle	Risco mitigado	Tratamento aplicado
Backup SQLite	Perda ou corrupção de bancos locais	Cópia dos bancos users, LGPD, segurança e contratações quando existentes.

Manifesto	Ausência de rastreabilidade	Registro de arquivos, tamanho, hash, data e status.
Hash SHA-256	Backup adulterado ou incompleto	Verificação posterior de integridade.
Restauração controlada	Erro operacional na recuperação	Confirmação explícita e cópia anterior antes da substituição.
Exportação MongoDB	Dependência exclusiva do banco remoto	Exportação opcional de coleção em JSONL.
Retenção	Acúmulo descontrolado de cópias	Remoção por quantidade máxima e idade.
Diagnóstico	Falta de visibilidade operacional	Checagem de diretório, bancos locais e último backup.

9. Parâmetros de configuração

Variável	Finalidade	Valor de referência
BACKUP_DIR	Diretório local de armazenamento dos backups.	backups
BACKUP_RETENTION_COUNT	Quantidade máxima de backups preservados.	10
BACKUP_RETENTION_DAYS	Idade máxima dos backups preservados.	30
MONGO_BACKUP_LIMIT	Limite de documentos exportados do MongoDB.	10000
USERS_DB_PATH	Caminho do banco local de usuários.	users.db
LGPD_DB_PATH	Caminho do banco local de solicitações LGPD.	lgpd_requests.db
SECURITY_DB_PATH	Caminho do banco local de eventos de segurança.	security_events.db
CONTRATACOES_DB_PATH	Caminho do banco local de contratações.	contratacoes.db

10. Critérios de aceite

Critério	Resultado esperado
Criar backup local	O sistema deve gerar ZIP no diretório configurado.
Manifesto do backup	O ZIP deve conter manifest.json.
Verificar integridade	O sistema deve comparar hashes calculados com os hashes do manifesto.
Restaurar banco	A restauração deve exigir confirmação explícita.
Preservar banco anterior	O arquivo substituído deve ser copiado antes da restauração.
Aplicar retenção	Backups antigos ou excedentes devem ser removidos.
Diagnóstico de continuidade	A interface deve informar bancos existentes, integridade e último backup.

11. Evidências de implementação

- Criação do módulo src/backup.py, responsável por backup, verificação, restauração, retenção e diagnóstico.
- Atualização do app.py com a área "Backup e continuidade de operação".
- Geração de backup local dos bancos SQLite configurados.
- Criação de manifesto manifest.json e cálculo de hash SHA-256.
- Verificação de integridade dos backups ZIP.
- Restauração controlada com cópia anterior.
- Exportação opcional de coleção MongoDB em JSONL.
- Atualização do .env.example e do .gitignore.

12. Arquivos impactados

- src/backup.py - novo módulo de backup e continuidade.
- app.py - integração da área de backup na interface Streamlit.
- .env.example - inclusão das variáveis de backup, retenção e bancos locais.
- .gitignore - exclusão de backups, bancos, logs e arquivos sensíveis.
- Documentação do Projeto/09_Backup_Continuidade_Operacao.md - documentação técnica resumida.

13. Limitações e melhorias futuras

- A solução atende ao escopo acadêmico, mas não substitui uma arquitetura completa de continuidade em produção.
- Em produção, seria recomendável usar armazenamento externo, criptografia dos backups, agendamento automático, alertas e testes periódicos de restauração.
- Também seria recomendável adotar infraestrutura redundante e monitoramento centralizado.

14. Conclusão

A quinta entrega adiciona ao PNCP Data Engine uma camada inicial de backup e continuidade de operação. A aplicação passa a gerar cópias verificáveis dos bancos locais, validar a integridade dos backups, restaurar bancos de forma controlada e oferecer diagnóstico operacional básico. Com isso, o projeto avança das camadas de segurança, privacidade e disponibilidade para uma estrutura mínima de recuperação e continuidade.